

TECHNICAL REFERENCE ARCHITECTURE

CivicPulse

End-to-End System Architecture, Asynchronous Data Pipelines, Protocols, and Detailed Execution Flows for Urban Anomaly Management

Event-Driven Architecture

RabbitMQ message broker & background notification workers for async SMS/email dispatching.

Jurisdictional Partitioning

State & area scoped multi-tenant data retrieval and role-based access control (RBAC).

Optimistic Concurrency Control

Version-backed (`_v`) atomic document updates preventing administrative race conditions.

Idempotent Ingestion

Sparse unique index validation preventing duplicate issue submissions on network retries.

1. Executive Summary & System Overview

CivicPulse is an enterprise-grade civic technology platform designed to bridge the operational gap between citizens and municipal administrations in India. The platform enables citizens to log urban anomalies (potholes, streetlighting failures, sanitation issues, water leakage) complete with photographic evidence, precise geolocation coordinates, and categorization tags.

Concurrently, municipal authorities access a localized, role-governed administrative dashboard that provides real-time triage, state/area level filtering, status tracking (*Pending* → *In Progress* → *Resolved*), and automated citizen notification dispatching.

Client Layer

- React 19 & Vite 6
- Tailwind CSS Utility Engine
- Axios JWT Interceptor
- i18next Internationalization

Gateway & API Layer

- Node.js / Express.js REST API
- Bcryptjs Password Hashing
- Express Rate Limiter
- Multer Attachment Handling

Async & Messaging

- RabbitMQ Message Broker
- Durable Notification Queues
- Background Consumer Worker
- SMS & Email Transports

2. Global Architectural Topology

The system follows a modern decoupled architecture. The frontend SPA communicates statelessly over HTTP/REST using JSON Web Tokens (JWT). The backend handles data persistence in MongoDB while delegating non-blocking side-effects (notifications) to RabbitMQ.



3. Detailed Architectural Execution Flows

Flow 1: Citizen Registration & Authentication Sequence

Ensures secure user onboarding, role determination (Citizen vs Admin), state/area scope assignment, and password hashing before returning a signed JWT token.

1

Client Credentials Submission

User submits username, email, phone, state, area, role, and password via the Signup page.

POST /api/auth/signup

2

Validation & Hashing

Backend checks for existing email/username collision, then generates a salt and hashes the password using `bcryptjs` (10 salt rounds).

authController.js • bcrypt.hash()

3

Database Storage & JWT Issuance

Saves user record to MongoDB. Signs a JWT token containing `id` , `username` , `role` , `state` , and `area` .

jsonwebtoken.sign()

4

Client AuthContext Initialization

Frontend stores JWT in `localStorage` . Axios interceptor automatically injects `Authorization: Bearer <token>` into all subsequent HTTP headers.

AuthContext.jsx • axios.js

Flow 2: Incident Reporting & Evidence Ingestion Pipeline

Handles multipart image upload, GPS coordinate parsing, idempotency validation to prevent duplicate tickets, and database record creation.

1 Idempotency & Payload Preparation

Frontend generates a UUID for `x-idempotency-key` and packages title, description, category, latitude, longitude, and image file into `FormData` .

```
Report.jsx • crypto.randomUUID()
```

2 Multier File Processing

Express middleware validates image MIME type (.jpg, .png), generates a timestamped unique filename, and saves the file to local disk (/uploads/).

middleware/upload.js • Multer

3 Idempotent MongoDB Save

Creates a new Report document with state/area copied from `req.user`. If a network retry re-sends the same idempotency key, MongoDB's sparse unique index throws code `11000`, returning a duplicate safety response without creating a second record.

UserController.js • createReport()

Flow 3: Administrative Triage & Dashboard Aggregation

Provides municipal administrators with real-time statistics (total, pending, active, resolved reports) for their specific jurisdiction using a single MongoDB aggregation query.

Flow 4: Optimistic Concurrency Control (OCC) for Status Updates

Prevents race conditions when multiple municipal administrators attempt to update the same anomaly report simultaneously.

1

Version-Aware Status Request

Admin sends new status (e.g., resolved) along with the current document version __v .

PUT /api/admin/report/:id/status { status, version: __v }

2

Atomic Find & Increment

Backend executes Report.findOneAndUpdate({ _id, __v: version }, { \$set: { status }, \$inc: { __v: 1 } }) .

adminController.js • OCC Check

3

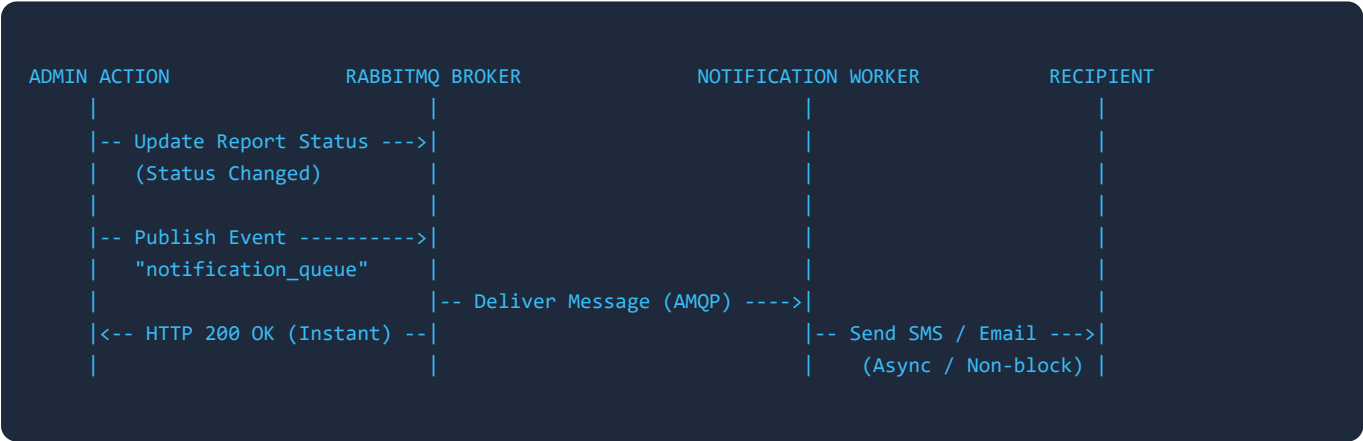
Conflict Resolution (409 Conflict)

If another admin updated the document first, the version match fails. The controller returns 409 Conflict , informing the user that the report was recently modified and requires a dashboard refresh.

HTTP 409 Conflict Response

Flow 5: Asynchronous Notification Pipeline (RabbitMQ Event Fan-Out)

Offloads non-critical side effects (sending SMS/email notifications to citizens) out of the main HTTP execution thread into a durable message queue.



4. Database Schema Specifications

4.1 User Document Schema

FIELD PATH	BSON TYPE	VALIDATION / INDEX	DESCRIPTION
<code>_id</code>	ObjectId	Primary Key	Unique user identifier.
<code>username</code>	String	Unique, Trim, Required	Citizen or Admin login handle.
<code>email</code>	String	Unique, Required, Index	User email address for alerts.
<code>password</code>	String	Required	Bcrypt hashed password string.
<code>role</code>	String	Enum: <code>['user', 'admin']</code>	System access control level.
<code>state</code>	String	Required, Index	Jurisdictional state partition.
<code>area</code>	String	Index	Local district/area partition.

4.2 Report Document Schema

FIELD PATH	BSON TYPE	VALIDATION / INDEX	DESCRIPTION
<code>_id</code>	ObjectId	Primary Key	Unique issue report identifier.
<code>title</code>	String	Required, Trim	Short summary of the urban anomaly.
<code>category</code>	String	Default: <code>'other'</code>	Category (roads, sanitation, water, electricity).
<code>status</code>	String	Enum: <code>['pending', 'in-progress', 'resolved']</code>	Resolution lifecycle status.
<code>user</code>	ObjectId	Ref: User, Index	Foreign key to submitting user.
<code>state</code> , <code>area</code>	String	Compound Index <code>{ state: 1, area: 1 }</code>	Geographic scoping fields.
<code>coordinates</code>	Object	<code>{ lat: Number, lng: Number }</code>	Spatial location parameters.
<code>idempotencyKey</code>	String	Sparse, Unique Index	Prevents duplicate request submission.
<code>__v</code>	Number	Default: <code>0</code>	Version key for Optimistic Concurrency Control.

5. Security, Performance & Resiliency Infrastructure

Rate Limiting Protection

Configured via `express-rate-limit`. Auth routes (`/api/auth/*`) are strictly capped at 15 requests per 15-minute window to prevent brute force login attempts. General API endpoints are capped at 100 requests per 15 minutes.

Compound Database Indexes

To ensure fast query performance across millions of rows, compound indexes on `{ state: 1, area: 1 }` and single field indexes on `user`, `email`, and `idempotencyKey` allow MongoDB to execute index scans (IXSCAN) instead of full collection scans (COLLSCAN).

Graceful RabbitMQ Fallback

In environments where RabbitMQ is offline or unconfigured, the backend logs a graceful warning and continues serving HTTP traffic without crashing, operating in mock notification mode.

6. Core API Endpoint Specification

METHOD	ENDPOINT PATH	ACCESS LEVEL	FUNCTIONALITY
POST	<code>/api/auth/signup</code>	Public	Register new user/admin with state & area.
POST	<code>/api/auth/login</code>	Public	Authenticate credentials & return JWT.
POST	<code>/api/user/report</code>	User / Admin	Submit report with image attachment & GPS.
GET	<code>/api/user/my-reports</code>	User / Admin	Fetch issues logged by current user.
GET	<code>/api/user/community</code>	User / Admin	Fetch community reports within user's state.
POST	<code>/api/user/report/:id/upvote</code>	User / Admin	Toggle community upvote on report.
GET	<code>/api/admin/reports</code>	Admin Only	Fetch all state/area reports for admin triage.
GET	<code>/api/admin/dashboard-stats</code>	Admin Only	Single-query aggregation for dashboard counters.
PUT	<code>/api/admin/report/:id/status</code>	Admin Only	Update report status with OCC version check.

© 2026 CivicPulse Platform Architecture Specification • Confidential & Proprietary